

BOUNCER VIP EXPRESS

Technical Development Plan & Investor Brief

Comprehensive architecture, filesystem design, technology stack, development directives and investor brief for the Bouncer VIP Security Platform and Express Supply Chain OS.

Document Type: Confidential Technical Development Plan

Version: 1.0 — Initial Release

Date: 21 April 2026

Company: Bouncer VIP Express (Pty) Ltd

Region: Greater Durban · KZN North Coast · South Coast

Classification: CONFIDENTIAL — NOT FOR DISTRIBUTION

TABLE OF CONTENTS

Document Structure

PART I — EXECUTIVE OVERVIEW

- 1.1 Company Vision & Mission
- 1.2 Business Model Overview
- 1.3 Platform Ecosystem Architecture
- 1.4 Market Context — South Africa

PART II — SYSTEM ARCHITECTURE

- 2.1 High-Level Architecture Diagram
- 2.2 Microservices Design Principles
- 2.3 Service Mesh & Communication Protocols
- 2.4 Data Architecture Strategy
- 2.5 Security Architecture

PART III — COMPREHENSIVE TECHNOLOGY STACK

- 3.1 Frontend & Mobile Stack
- 3.2 Backend Services Stack (Polyglot)
- 3.3 Infrastructure & DevOps Stack
- 3.4 Blockchain Infrastructure (Polygon CDK)
- 3.5 AI & Machine Learning Stack
- 3.6 Biometric & Facial Recognition Stack
- 3.7 Messaging & Event Streaming
- 3.8 Observability & Monitoring Stack

PART IV — FILESYSTEM & PROJECT STRUCTURE

- 4.1 Monorepo Root Layout
- 4.2 Bouncer VIP Platform Services
- 4.3 Express Platform Services
- 4.4 Shared Libraries & Packages
- 4.5 Infrastructure-as-Code Structure
- 4.6 Smart Contract Structure

PART V — BOUNCER VIP PLATFORM — CORE MODULES

- 5.1 CRM Module
- 5.2 Scheduling Engine
- 5.3 On-Demand Booking Service
- 5.4 Digital Monitoring & Live Operations
- 5.5 ERP — Payroll, Billing & Compliance
- 5.6 Workflow Automation Engine
- 5.7 Client Portal
- 5.8 Guard Mobile Application
- 5.9 Analytics & Intelligence Dashboard

PART VI — BIOMETRIC CLOCK-IN/OUT & PENALTY ENGINE

- 6.1 Facial Recognition Architecture
- 6.2 Geofencing & GPS Verification
- 6.3 Liveness Detection
- 6.4 Penalty Engine & Disciplinary Workflow
- 6.5 Biometric Data Privacy & Compliance

PART VII — BLOCKCHAIN INFRASTRUCTURE

- 7.1 Polygon CDK App-Chain Design
- 7.2 Smart Contract Architecture
- 7.3 On-Chain Record Types
- 7.4 Validator Configuration
- 7.5 Bridge & Interoperability

PART VIII — AI RECRUITMENT & ONBOARDING PIPELINE

- 8.1 Recruitment Funnel Architecture
- 8.2 AI Pre-Screening Engine
- 8.3 Async Video Interview Analysis
- 8.4 Credential Verification System
- 8.5 Digital Onboarding Flow

PART IX — EXPRESS — SUPPLY CHAIN OS

- 9.1 Express Platform Overview
- 9.2 Warehouse Management System
- 9.3 Cold Chain IoT Architecture
- 9.4 AI Demand Forecasting Engine
- 9.5 Logistics & Fleet Management
- 9.6 Last-Mile Delivery (B2C)
- 9.7 Supplier Integration Network
- 9.8 Express ERP & Billing

PART X — INFRASTRUCTURE & DEVOPS

- 10.1 AWS EC2 Deployment Topology
- 10.2 Coolify OSS Orchestration
- 10.3 CI/CD Pipeline Architecture
- 10.4 Secrets Management — Infisical OSS
- 10.5 Traefik Ingress & TLS
- 10.6 Kong OSS API Gateway
- 10.7 NATS JetStream Event Bus
- 10.8 Grafana Observability Stack

PART XI — DEVELOPMENT DIRECTIVES

- 11.1 Engineering Principles
- 11.2 Coding Standards per Language
- 11.3 API Design Standards
- 11.4 Database-per-Service Strategy
- 11.5 Testing Strategy

11.6 Security Development Lifecycle

PART XII — PHASED ROLLOUT ROADMAP

- 12.1 Phase 1 — Core Security Launch
- 12.2 Phase 2 — Biometrics & Blockchain
- 12.3 Phase 3 — Express Launch
- 12.4 Phase 4 — National Expansion

PART XIII — INVESTOR BRIEF

- 13.1 Investment Thesis
- 13.2 Market Opportunity
- 13.3 Revenue Model & Projections
- 13.4 Capital Requirements
- 13.5 Funding Strategy — Banks & DFIs
- 13.6 Risk Analysis & Mitigation
- 13.7 Competitive Moat
- 13.8 Exit Strategy

PART I

EXECUTIVE OVERVIEW

Vision, mission, business model and market context

§ 1.1

Company Vision & Mission

Bouncer VIP Express (Pty) Ltd is a South African technology-first security and hospitality supply chain company headquartered in the Greater Durban region, KwaZulu-Natal. The company operates two interconnected service pillars: the Bouncer VIP Platform — an AI-powered enterprise orchestration system for professional security deployment — and Express, a cloud-native supply chain operating system purpose-built for the South African nightlife, events, hospitality and fine dining industries.

Vision

To become the defining operating infrastructure for the South African nightlife and events economy — making every venue safer, every supply chain smarter, and every operation verifiable through immutable blockchain technology.

Mission

To deploy professional, PSIRA-compliant security personnel with unmatched reliability, verified by biometric and blockchain technology, while simultaneously providing venues with an AI-driven supply chain OS that eliminates procurement friction, cold chain inefficiency, and last-mile delivery failure.

§ 1.2

Business Model Overview

The company operates a dual-flywheel model. The security services business (Bouncer VIP) generates immediate recurring revenue through subscription-based bouncer deployment, acting as the commercial

foundation and the proof-of-market that unlocks capital for the Express infrastructure build-out.

Pillar	Revenue Stream	Mechanism	Timeline
Bouncer VIP	Security as a Service	Venue subscriptions (daily/weekly/monthly)	Immediate
Bouncer VIP	Platform SaaS	CRM, scheduling, ERP licensing	Month 3+
Bouncer VIP	Biometric Verification	Premium tier add-on for verified deployments	Month 6+
Express	Storage as a Service	Monthly cold/dry storage subscriptions per venue	Month 8+
Express	Logistics Fees	Per-delivery charges (scheduled + on-demand)	Month 8+
Express	Procurement Margin	5–12% margin on bulk supplier purchases	Month 10+
Express	B2C Delivery Commission	18–25% on consumer orders via Express App	Month 12+
Express	SaaS Platform Fee	ERP integration + demand forecasting tier	Month 8+

§ 1.3

Platform Ecosystem Architecture

Both platforms are unified under a single enterprise dashboard accessible to management, with role-segregated portals for venue clients, security personnel, logistics staff, and warehouse operators. The underlying architecture is a polyglot microservices system connected via gRPC for synchronous communication and NATS JetStream for asynchronous event streaming, with all critical state changes anchored to a dedicated Polygon CDK application blockchain.

Primary User Roles Across the Ecosystem:

- **Super Admin** — Full system access, blockchain node oversight, financial controls
- **Operations Manager** — Deployment scheduling, incident management, fleet oversight
- **Dispatch Officer** — Real-time bouncer allocation, on-demand request handling
- **Venue Client (Manager)** — Portal access, booking, stock management, invoice review
- **Security Officer (Bouncer)** — Mobile app — shifts, biometric clock-in, incident reports
- **Warehouse Operator** — Inventory management, cold chain monitoring, dispatch coordination
- **Driver / Rider** — Mobile delivery app, route assignment, proof-of-delivery capture
- **Compliance Officer** — PSIRA tracking, blockchain audit trails, disciplinary records
- **HR / Recruiter** — AI recruitment pipeline, onboarding management, training tracking
- **Finance Officer** — ERP, payroll, invoicing, DFI reporting, bank reconciliation

§ 1.4

Market Context — South Africa

South Africa's private security industry is one of the largest globally, with over 500,000 registered officers and an industry valued at approximately R65 billion annually. Despite this scale, the sector remains highly fragmented, technologically underdeveloped, and compliance-inconsistent — particularly in the nightlife and events sub-vertical.

Metric	Value	Source
Private Security Market Size (SA)	~R65 billion/year	PSIRA Industry Report
Registered Security Officers (SA)	500,000+	PSIRA 2024
Active Nightlife Venues — Greater Durban	300–500 venues	Market Estimate
Active Venues — KZN North Coast	150–200 venues	Market Estimate
Active Venues — KZN South Coast	80–120 venues	Market Estimate
Events/Month — KZN Peak Season	200–400 events	Market Estimate
Cold Chain Logistics Market (SA)	~R12 billion/year	Industry Analysis
Food & Beverage Delivery Market (SA)	R8–10 billion/year	Statista SA 2024

The Greater Durban region — spanning Florida Road, Umhlanga, Point Road, and the entertainment corridors of eThekwin Metro — represents an immediate addressable market of 300–500 nightlife and events venues, with zero incumbent competitors offering a technology-first, blockchain-verified, AI-orchestrated security deployment platform.

§ 2.2

Microservices Design Principles

Single Responsibility

Each service owns one bounded context. No service directly reads another service's database.

Database-per-Service

Services maintain isolated data stores. PostgreSQL for relational, MongoDB for documents, Redis for cache, InfluxDB for time-series telemetry.

API-First Design

All service interfaces defined in Protocol Buffer (.proto) files before implementation begins. OpenAPI specs generated for REST endpoints.

Event-Driven by Default

State changes emit domain events to NATS JetStream. Consumers process events independently, enabling loose coupling.

Idempotency Guaranteed

All write operations implement idempotency keys. NATS consumers use exactly-once semantics via JetStream consumer groups.

Circuit Breaking

All gRPC service clients wrapped with circuit breaker (Resilience4j / Go's sony/gobreaker). Fallback strategies defined per service.

Observability Built-In

Every service emits structured logs (JSON), metrics (Prometheus), and distributed traces (OpenTelemetry → Tempo) from day one.

Zero-Trust Internal Network

mTLS enforced between all internal services. Service identity managed via SPIFFE/SPIRE.

§ 2.3

Service Mesh & Communication Protocols

Synchronous — gRPC + Protocol Buffers

All synchronous inter-service communication uses gRPC with Protocol Buffer schemas. This provides strongly-typed contracts, automatic client/server code generation across all supported languages (TypeScript, Go, Python, Java, Rust, Elixir), and significantly better performance than REST for internal calls.

```
# Example: Shift Service Proto Definition
syntax = "proto3";
package bouncer.shift.v1;

service ShiftService {
  rpc CreateShift(CreateShiftRequest) returns (ShiftResponse);
  rpc AssignOfficer(AssignOfficerRequest) returns (ShiftResponse);
  rpc ClockIn(ClockInRequest) returns (ClockInResponse);
  rpc ClockOut(ClockOutRequest) returns (ClockOutResponse);
  rpc GetActiveShifts(GetActiveShiftsRequest) returns (stream ShiftEvent);
}

message ClockInRequest {
  string officer_id = 1;
  string shift_id = 2;
  BiometricPayload biometric = 3;
  GeoLocation location = 4;
  int64 timestamp_unix = 5;
}
```

Asynchronous — NATS JetStream

All asynchronous, fire-and-forget, and event-sourcing patterns use NATS JetStream. Streams are organised by domain with wildcard subjects for granular consumer filtering.

NATS Subject	Description	Publishers → Consumers
deployments.created	New shift deployment created	Scheduling → Monitoring, Blockchain
deployments.clockin	Officer biometric clock-in verified	Biometric Svc → Blockchain, ERP
deployments.clockout	Officer clock-out recorded	Biometric Svc → Blockchain, ERP, Penalty
incidents.reported	Incident logged by officer/manager	Monitoring → Blockchain, CRM, Alert
penalty.triggered	Late arrival/no-show flagged	Penalty Engine → HR, ERP, Dispatch
orders.created	Express stock order placed	WMS → Procurement, Logistics
delivery.dispatched	Fleet vehicle dispatched	Logistics → Tracking, Client Portal
delivery.completed	Delivery confirmed + PoD captured	Logistics → Blockchain, Billing
telemetry.iot	Cold chain sensor readings	IoT Gateway → InfluxDB, Alerts
recruitment.applied	New officer application submitted	Recruitment → AI Screener

§ 2.4

Data Architecture Strategy

Each microservice maintains its own data store, selected based on the access patterns and consistency requirements of that domain. No cross-service database queries are permitted; all cross-service data access is via API calls or event consumption.

Service	Database	Rationale
CRM Service	PostgreSQL 16	Relational — venue contracts, pipelines, contacts
Scheduling Service	PostgreSQL 16	Relational — shifts, rosters, officer assignments
Officer Profiles	PostgreSQL 16	Relational — PSIRA records, biometric hashes, conduct
On-Demand Booking	PostgreSQL 16 + Redis	Transactional + real-time availability cache
Incident Reports	MongoDB 7	Document — flexible incident schema, media attachments
Monitoring / Live Ops	Redis + PostgreSQL	Real-time state cache + persistent records
ERP / Payroll	PostgreSQL 16	Relational — financial precision, audit trails
Analytics Engine	ClickHouse	OLAP columnar — high-speed aggregation queries
IoT Cold Chain	InfluxDB 2	Time-series — sensor readings at 30s intervals
WMS Inventory	PostgreSQL 16	Relational — stock ledger, FIFO/FEFO tracking
AI / ML Models	MinIO (OSS S3)	Object store — model artefacts, training datasets
Media / Documents	MinIO (OSS S3)	Object store — incident photos, contracts, IDs
Session / Auth Cache	Redis 7	JWT token store, session management
Search	Meilisearch (OSS)	Full-text — officer search, venue search, docs
Blockchain State	Polygon CDK (EVM)	Immutable — shift records, compliance, incidents

§ 2.5

Security Architecture

Authentication & Authorisation

Authentication is handled by Keycloak OSS acting as the central identity provider. All user-facing applications (web dashboard, guard app, client portal, express app) authenticate via OAuth2 / OpenID Connect. Internal services use mTLS for mutual authentication. Role-Based Access Control (RBAC) is enforced at the API Gateway layer and re-validated at each service boundary.

- JWT access tokens (15-minute expiry) + rotating refresh tokens (7-day expiry)
- All secrets managed via Infisical OSS with AWS Secrets Manager as backup layer
- Database credentials rotated automatically every 30 days via Infisical
- TLS 1.3 enforced on all external endpoints via Traefik with Let's Encrypt

- mTLS enforced on all internal service-to-service communication
- SPIFFE/SPIRE for workload identity — no static service credentials
- API rate limiting enforced at Kong OSS gateway (per-client, per-route)
- OWASP Top 10 checklist applied at every service during security review
- Biometric data: only cryptographic hashes stored — raw biometrics never persisted
- Blockchain records: cryptographically signed by the originating service key
- POPIA (Protection of Personal Information Act) compliance by design
- GDPR-aligned data residency — all data stored in AWS af-south-1 (Cape Town)

PART III

COMPREHENSIVE TECHNOLOGY STACK

Every language, framework, library and tool in the ecosystem

§ 3.1

Frontend & Mobile Stack

Technology	Language	Purpose
Next.js 14	TypeScript	Corporate landing page, management dashboard, client portal — SSR + ISR
React 18	TypeScript	SPA components within Next.js — dashboard widgets, live maps
Tailwind CSS	—	Utility-first styling across all web interfaces
shadcn/ui	TypeScript	Accessible component library built on Radix UI primitives
Maplibre GL JS	TypeScript	Open-source map rendering for live officer/fleet tracking
Recharts	TypeScript	Analytics charts — deployment density, revenue, performance
React Query	TypeScript	Server state management, caching, background refetching
Zustand	TypeScript	Lightweight client state management (UI state, auth state)
React Hook Form	TypeScript	Form management with Zod schema validation
Socket.io Client	TypeScript	Real-time dashboard updates (officer locations, incidents)
React Native 0.74	TypeScript	Guard mobile application (iOS + Android)
Flutter 3.x	Dart	Express consumer app + B2C delivery tracking (iOS + Android)
Expo	TypeScript	Guard app build tooling, OTA updates, device API access
React Native Maps	TypeScript	In-app maps for guard navigation and shift location

Technology	Language	Purpose
TensorFlow Lite	TypeScript/C++	On-device facial recognition inference (guard app)
MediaPipe Face	C++/WASM	Face detection and landmark extraction for biometrics

§ 3.2

Backend Services Stack — Polyglot Architecture

The platform is intentionally polyglot, selecting each language based on the performance and productivity characteristics best suited to each service domain. All services communicate via Proto-defined gRPC contracts, making language selection transparent to consumers.

Language / Runtime	Services & Responsibilities
TypeScript / Node.js	CRM Service, Client Portal API, Notification Service, Webhook Gateway
Go 1.22	On-Demand Dispatch, Real-Time Monitoring, API Gateway Custom Plugins, Fleet Tracking
Python 3.12 (FastAPI)	AI Recruitment Engine, Demand Forecasting, Analytics API, ML Model Serving
Java 21 (Spring Boot)	ERP Core, Payroll Engine, Financial Ledger, PSIRA Compliance Service
Rust	Biometric Verification Service, Cryptographic Signing, High-Throughput Event Processor
Elixir (Phoenix)	Scheduling Engine, Real-Time Presence (WebSocket), Pub/Sub Coordination
Solidity 0.8.x	Smart Contracts — ShiftRegistry, IncidentLedger, ComplianceVault, DeliveryPoD
Python (Celery)	Async task workers — report generation, batch payroll, email dispatch
Go	IoT Gateway — ingesting cold chain sensor data into InfluxDB

§ 3.3

Infrastructure & DevOps Stack

Technology	Role
AWS EC2 (t3.medium)	Primary compute — application services, initially consolidated
AWS EC2 (c5.xlarge)	Compute-optimised instances for ML model serving and biometric processing
AWS EC2 (r5.large)	Memory-optimised for PostgreSQL primary nodes and Redis clusters
AWS S3 / af-south-1	Backup storage for MinIO, database snapshots, log archives
AWS VPC	Network isolation — private subnets for databases, public for ingress only
Coolify OSS	Self-hosted PaaS orchestration — container lifecycle, deploy webhooks
Docker + Docker Compose	Service containerisation — all services ship as OCI images

Technology	Role
Traefik v3	Ingress controller, TLS termination (Let's Encrypt), load balancing
Kong OSS	External API gateway — auth, rate limiting, logging, routing
Keycloak OSS	Identity provider — OAuth2/OIDC, RBAC, SSO across all portals
Infisical OSS	Secrets management — env vars, DB credentials, API keys
AWS Secrets Manager	Backup secrets layer for critical production credentials
GitHub Actions	CI/CD pipeline — test, lint, build, push, deploy via Coolify webhooks
Terraform	Infrastructure-as-Code for AWS resources (VPC, EC2, SGs, IAM)
Ansible	Server provisioning, Coolify setup, system configuration management
MinIO OSS	Self-hosted S3-compatible object storage for media and artefacts
Harbor OSS	Private container registry — Docker image storage and scanning
Trivy	Container image vulnerability scanning in CI pipeline
SPIFFE/SPIRE	Workload identity — mTLS certificate issuance for service mesh

§ 3.4

Blockchain Infrastructure — Polygon CDK

Bouncer VIP Express operates a dedicated EVM-compatible application blockchain built with Polygon CDK (Chain Development Kit). This sovereign app-chain provides immutable record-keeping for all critical operational events — shift deployments, biometric clock-ins, incident reports, compliance certificates and delivery proof-of-delivery.

Component	Purpose
Polygon CDK	App-chain framework — EVM-compatible, sovereign validator set
Solidity 0.8.x	Smart contract language for all on-chain record types
Hardhat	Smart contract development, testing, and deployment framework
ethers.js v6	TypeScript/JS library for blockchain interaction in backend services
OpenZeppelin Contracts	Audited base contracts — AccessControl, Pausable, Upgradeable
The Graph (OSS)	Subgraph indexing for efficient on-chain data queries
Polygon AggLayer	Future interoperability bridge to Polygon PoS mainnet
Foundry	High-performance Solidity testing framework (Forge, Cast, Anvil)
Slither	Static analysis for smart contract security auditing
Custom Gas Token	Internal BVX utility token — zero-cost internal transactions

§ 3.5

AI & Machine Learning Stack

Technology	Purpose
Python 3.12 + FastAPI	ML model serving API — REST endpoints for all AI features
PyTorch 2.x	Primary deep learning framework — facial recognition, NLP models
scikit-learn	Classical ML — demand forecasting regression, anomaly detection
Prophet (Meta)	Time-series demand forecasting for Express inventory planning
Transformers (HuggingFace)	NLP for recruitment video transcript analysis, risk scoring
OpenCV	Computer vision preprocessing — image normalisation, face detection
DeepFace	Facial recognition framework wrapping multiple model backends
FaceTec SDK / Alternative	Liveness detection — 3D depth analysis anti-spoofing
Celery + Redis	Async ML task queue — batch inference, model retraining jobs
MLflow	ML experiment tracking, model versioning, artefact management
Ray Serve	Scalable model serving for high-throughput inference endpoints
Pandas + NumPy	Data processing pipeline for analytics and feature engineering
Jupyter (internal)	Data science workspace for model development and analysis

§ 3.6

Biometric & Facial Recognition Stack

Biometric verification is implemented as a privacy-preserving, on-device-first system. Raw biometric data (face images) is processed locally on the device and only a cryptographic hash of the verified identity is transmitted to backend services and anchored on-chain. No raw biometric images are stored on company infrastructure.

- TensorFlow Lite (on-device) — Face embedding model runs entirely on the officer's device
- MediaPipe Face Mesh — 468 facial landmark detection for precise geometry capture
- FaceTec / Alternative OSS — 3D liveness detection preventing photo and video spoofing
- ArcFace model — State-of-the-art face recognition achieving 99.7%+ LFW accuracy
- Geofencing layer — GPS coordinates verified against venue polygon (±50m tolerance)
- Multi-factor confirmation — face match + GPS + device fingerprint all required for clock-in
- Fallback — fingerprint biometric via device TouchID/Fingerprint API if face recognition unavailable
- Backend (Rust service) — receives biometric confidence score + cryptographic hash, never raw data
- Hash construction — BLAKE3(face_embedding + officer_id + timestamp + shift_id)

- On-chain anchoring — biometric hash + geo hash + timestamp written to ShiftRegistry contract

§ 3.7

Messaging & Event Streaming

NATS JetStream serves as the central nervous system of the platform, handling all asynchronous communication between services. JetStream provides persistence, replay, exactly-once delivery, and consumer group semantics on top of the core NATS protocol.

```
# NATS JetStream Stream Configuration
streams:
  - name: DEPLOYMENTS
    subjects: ["deployments.>"]
    storage: file           # Persistent to disk
    retention: limits       # Keep last 30 days
    max_age: 2592000s       # 30 days in seconds
    replicas: 3             # 3-node NATS cluster
    max_msg_size: 1MB

  - name: INCIDENTS
    subjects: ["incidents.>"]
    storage: file
    retention: limits
    max_age: 7776000s       # 90 days (legal requirement)
    replicas: 3

  - name: IOT_TELEMETRY
    subjects: ["telemetry.coldchain.>"]
    storage: file
    retention: limits
    max_age: 604800s        # 7 days (high volume)
    replicas: 2

  - name: ORDERS
    subjects: ["orders.>", "delivery.>"]
    storage: file
    retention: limits
    max_age: 7776000s       # 90 days
    replicas: 3
```

§ 3.8

Observability & Monitoring Stack

Technology	Purpose
Prometheus	Metrics collection — all services expose /metrics endpoint
Grafana OSS	Dashboards — operational KPIs, service health, business metrics
Grafana Loki	Log aggregation — structured JSON logs from all services
Grafana Tempo	Distributed tracing — OpenTelemetry traces for request flows

Technology	Purpose
OpenTelemetry SDK	Instrumentation library — traces, metrics, logs in all services
Grafana Alerting	Alert rules for SLA breaches, error rate spikes, cold chain alarms
cAdvisor	Container resource metrics — CPU, memory, network per container
node_exporter	Host-level metrics — EC2 instance health
Uptime Kuma	External uptime monitoring — public-facing endpoint availability
PagerDuty / Grafana OnCall	On-call alerting and escalation policies

PART IV

FILESYSTEM & PROJECT STRUCTURE

Complete monorepo layout, service directories and IaC structure

§ 4.1

Monorepo Root Layout

The entire Bouncer VIP Express codebase is managed as a monorepo using Turborepo for build orchestration and pnpm workspaces for JavaScript/TypeScript dependency management. Non-JS services (Go, Python, Java, Rust, Elixir, Solidity) are co-located as workspace members with their own build tooling.

```
bouncer-vip-express/
├── apps/
│   ├── web/
│   ├── guard-app/
│   └── express-app/
├── services/
│   ├── bouncer/
│   └── express/
├── packages/
│   ├── proto/
│   ├── ui/
│   ├── types/
│   ├── config/
│   ├── utils/
│   └── contracts/
│       ├── src/
│       ├── test/
│       ├── scripts/
│       └── deployments/
├── infra/
│   ├── terraform/
│   ├── ansible/
│   ├── coolify/
│   └── nats/
└── .github/

# Deployable applications
# Next.js management dashboard + landing page
# React Native guard mobile application
# Flutter consumer + B2C delivery app
# Backend microservices
# Bouncer VIP core services
# Express supply chain services
# Shared libraries (TypeScript)
# Protocol Buffer definitions (all services)
# Shared React component library
# Shared TypeScript type definitions
# Shared ESLint, Prettier, TS configs
# Shared utility functions
# Solidity smart contracts
# Contract source files
# Hardhat + Foundry tests
# Deployment + migration scripts
# Deployed contract addresses per network
# Infrastructure-as-Code
# AWS resource definitions
# Server provisioning playbooks
# Coolify service definitions
# NATS JetStream configuration
# GitHub Actions CI/CD workflows
```

§ 4.2

Bouncer VIP Platform Services

```
services/bouncer/
■■■■ crm-service/                                # TypeScript / Node.js
■   ■■■■ src/
■   ■   ■■■■ entities/                          # Venue, Contact, Contract, Pipeline
■   ■   ■■■■ repositories/                      # PostgreSQL data access layer
■   ■   ■■■■ grpc/                             # gRPC server handlers
■   ■   ■■■■ events/                           # NATS event publishers/consumers
■   ■   ■■■■ config/                           # Env, logger, db connection
■   ■■■■ Dockerfile
■   ■■■■ package.json
■■■■ scheduling-service/                        # Elixir / Phoenix
■   ■■■■ lib/
■   ■   ■■■■ scheduling/                        # Shift GenServer, Roster Manager
■   ■   ■■■■ presence/                         # Phoenix Presence for live officer state
■   ■   ■■■■ grpc/                             # gRPC handler (grpc-elixir)
■   ■   ■■■■ events/                           # NATS publisher (gnat)
■   ■■■■ mix.exs
■   ■■■■ Dockerfile
■■■■ dispatch-service/                         # Go
■   ■■■■ internal/
■   ■   ■■■■ handler/                          # gRPC handlers
■   ■   ■■■■ dispatcher/                      # Real-time officer matching algorithm
■   ■   ■■■■ geofence/                        # Venue polygon geofencing
■   ■   ■■■■ nats/                            # JetStream publisher/consumer
■   ■■■■ cmd/server/main.go
■   ■■■■ Dockerfile
■■■■ monitoring-service/                       # Go
■   ■■■■ internal/
■   ■   ■■■■ tracker/                         # Live GPS state machine
■   ■   ■■■■ incident/                       # Incident CRUD + escalation
■   ■   ■■■■ websocket/                      # Live dashboard WebSocket hub
■   ■   ■■■■ blockchain/                     # Write incident hashes on-chain
■   ■■■■ Dockerfile
■■■■ erp-service/                             # Java 21 / Spring Boot
■   ■■■■ src/main/java/com/bouncervip/erp/
■   ■   ■■■■ payroll/                         # Payroll calculation engine
■   ■   ■■■■ billing/                         # Invoice generation, subscription mgmt
■   ■   ■■■■ compliance/                     # PSIRA tracking, certificate management
■   ■   ■■■■ ledger/                         # Financial double-entry ledger
■   ■■■■ Dockerfile
■■■■ biometric-service/                       # Rust
■   ■■■■ src/
■   ■   ■■■■ grpc/                           # tonic gRPC server
■   ■   ■■■■ verifier/                       # Biometric hash validation
■   ■   ■■■■ geofence/                       # Coordinate validation
■   ■   ■■■■ blockchain/                     # ethers-rs on-chain anchoring
■   ■■■■ Cargo.toml
■   ■■■■ Dockerfile
■■■■ penalty-service/                         # TypeScript / Node.js
■   ■■■■ src/
■   ■   ■■■■ engine/                          # Lateness scoring, tier calculation
■   ■   ■■■■ workflows/                      # Disciplinary workflow state machines
■   ■   ■■■■ events/                         # Consumes deployments.clockin from NATS
■   ■■■■ Dockerfile
■■■■ recruitment-service/                     # Python / FastAPI
■   ■■■■ app/
```


§ 4.3

Express Platform Services

services/express/	
■■■■ wms-service/	# TypeScript / Node.js
■ ■■■■ src/	
■ ■ ■■■■ inventory/	# Stock ledger (FIFO/FEFO), lot tracking
■ ■ ■■■■ warehouse/	# Bay management, zone allocation
■ ■ ■■■■ inbound/	# Goods receipt, supplier PO matching
■ ■ ■■■■ outbound/	# Pick, pack, dispatch coordination
■ ■■■■ Dockerfile	
■■■■ cold-chain-service/	# Go
■ ■■■■ internal/	
■ ■ ■■■■ iot_gateway/	# MQTT ingest from IoT sensors
■ ■ ■■■■ telemetry/	# InfluxDB write layer
■ ■ ■■■■ alerts/	# Temperature breach detection + NATS publish
■ ■ ■■■■ compliance/	# Cold chain audit trail
■ ■■■■ Dockerfile	
■■■■ procurement-service/	# TypeScript / Node.js
■ ■■■■ src/	
■ ■ ■■■■ suppliers/	# Supplier profile + contract management
■ ■ ■■■■ orders/	# Purchase order lifecycle
■ ■ ■■■■ pricing/	# Negotiated rate management
■ ■ ■■■■ integrations/	# SAB, Heineken, Distell API connectors
■ ■■■■ Dockerfile	
■■■■ forecasting-service/	# Python / FastAPI
■ ■■■■ app/	
■ ■ ■■■■ models/	# Prophet + scikit-learn demand models
■ ■ ■■■■ training/	# Celery workers for model retraining
■ ■ ■■■■ predictions/	# Real-time stock level predictions
■ ■ ■■■■ recommendations/	# Auto-PO generation recommendations
■ ■■■■ Dockerfile	
■■■■ logistics-service/	# Go
■ ■■■■ internal/	
■ ■ ■■■■ fleet/	# Vehicle & driver management
■ ■ ■■■■ routing/	# OSRM route optimisation integration
■ ■ ■■■■ dispatch/	# Delivery assignment algorithm
■ ■ ■■■■ tracking/	# Real-time vehicle GPS state
■ ■■■■ Dockerfile	
■■■■ lastmile-service/	# Go
■ ■■■■ internal/	
■ ■ ■■■■ consumer_orders/	# B2C order intake + management
■ ■ ■■■■ rider_dispatch/	# Rider assignment algorithm
■ ■ ■■■■ restaurant/	# Venue menu management
■ ■ ■■■■ pod/	# Proof-of-delivery capture + blockchain
■ ■■■■ Dockerfile	
■■■■ express-erp-service/	# Java 21 / Spring Boot
■■■■ src/main/java/com/bouncervip/express/erp/	
■ ■■■■ billing/	# Storage, logistics, SaaS invoicing
■ ■■■■ procurement/	# Supplier payment management
■ ■■■■ reporting/	# DFI-ready financial reporting
■■■■ Dockerfile	

§ 4.4

Shared Packages

```
packages/
  proto/                                # Single source of truth for all gRPC contracts
    bouncer/
      crm/v1/crm.proto
      shift/v1/shift.proto
      biometric/v1/biometric.proto
      incident/v1/incident.proto
      erp/v1/erp.proto
    express/
      wms/v1/wms.proto
      logistics/v1/logistics.proto
      forecasting/v1/forecasting.proto
    gen/                                # Auto-generated client stubs (all languages)
      ts/
      go/
      python/
      java/
      rust/
  ui/                                    # Shared React component library
    src/components/                     # Button, Card, Table, Badge, Modal, etc.
    src/charts/                         # Recharts wrappers with brand styling
    src/maps/                           # Maplibre components
  types/                                # Shared TypeScript domain types
  config/
    eslint-config/
    prettier-config/
    tsconfig/
```

§ 4.5

Infrastructure-as-Code Structure

```
infra/
  terraform/
    modules/
      vpc/                             # VPC, subnets, IGW, NAT Gateway
      ec2/                             # Instance definitions, security groups
      rds/                             # PostgreSQL RDS (if migrating from self-hosted)
      s3/                              # Backup buckets, log archives
      iam/                             # IAM roles, policies for EC2 + Secrets Manager
    environments/
      staging/                          # main.tf, variables.tf, outputs.tf
      production/                      # main.tf, variables.tf, outputs.tf
      backend.tf                       # Terraform state backend (S3 + DynamoDB lock)
  ansible/
    playbooks/
      provision.yml                   # Base server setup (Docker, Coolify, firewall)
      nats-cluster.yml               # 3-node NATS JetStream cluster setup
      monitoring.yml                 # Prometheus, Grafana Loki/Tempo stack
      polygon-node.yml               # Polygon CDK validator node setup
    inventory/
      staging.ini
      production.ini
  nats/
    server.conf                      # NATS server configuration
    jetstream.conf                   # Stream and consumer definitions
    cluster.conf                     # 3-node cluster configuration
```

§ 4.6

Smart Contract Structure

```
contracts/
  src/
    core/
      ShiftRegistry.sol      # Immutable shift deployment records
      IncidentLedger.sol     # Tamper-proof incident reports
      ComplianceVault.sol   # PSIRA cert hashes, expiry tracking
      biometric/
        BiometricAnchor.sol # Clock-in/out biometric hash anchoring
      express/
        DeliveryPoD.sol     # Proof-of-delivery immutable records
        ColdChainAudit.sol  # Temperature excursion audit trail
      access/
        BVEAccessControl.sol # Role-based access (extends OZ AccessControl)
      interfaces/
        IShiftRegistry.sol
        IIncidentLedger.sol
        IDeliveryPoD.sol
    test/
      hardhat/               # Hardhat TypeScript tests
      foundry/              # Forge Solidity tests (fuzz testing)
    scripts/
      deploy.ts              # Hardhat deploy scripts
      verify.ts              # Contract verification on block explorer
    deployments/
      testnet.json           # Testnet deployed addresses
      mainnet.json           # Production deployed addresses
    hardhat.config.ts
    foundry.toml
```

PART V

BOUNCER VIP — CORE MODULES

Detailed specification of all nine core platform modules

§ 5.1

CRM Module

The CRM module manages the complete venue client lifecycle from initial lead capture through contract signing, renewal tracking, and client health scoring. Built in TypeScript/Node.js with PostgreSQL, it exposes a gRPC interface consumed by the scheduling engine, ERP service, and client portal.

- Venue pipeline with customisable stages (Lead → Qualified → Demo → Proposal → Active → Churned)
- Contract lifecycle management — version history, renewal dates, SLA terms, rate cards
- Contact management — multiple stakeholders per venue with role classification
- Client health scoring — automated score based on payment history, shift continuity, incident frequency
- Renewal alert engine — 60/30/14-day automated notifications to account managers
- Integration with ERP for revenue-per-venue analytics
- Activity timeline — every interaction, booking, incident and invoice linked to the venue record
- Bulk communications — targeted email/SMS campaigns to venue segments

§ 5.2

Scheduling Engine

The scheduling engine is the operational core of the platform, built in Elixir/Phoenix for its superior concurrency model and real-time pub/sub capabilities via Phoenix Presence. It manages the complete lifecycle of a shift — creation, officer assignment, roster conflict detection, and real-time status tracking.

- Visual drag-and-drop shift builder with calendar week/month/day views
- Officer availability matrix — officers set availability via Guard App, engine respects constraints

- Conflict detection — automatic alerting when an officer is double-booked or under rest period
- Venue blacklist/whitelist — venues can block or request specific officers
- Auto-assignment algorithm — matches available officers by proximity, rating, venue familiarity
- Bulk scheduling — upload weekly/monthly rosters via CSV or template
- Shift template library — recurring patterns for regular venues saved as reusable templates
- Real-time presence tracking — Elixir GenServer tracks officer availability state changes
- Emergency replacement workflow — if officer no-shows, system auto-identifies nearest available backup
- Shift confirmation flow — officers confirm shifts 24h in advance via Guard App push notification

§ 5.3

On-Demand Booking Service

Venues access on-demand booking via their client portal. A request triggers a real-time matching algorithm (Go service) that identifies available, nearby, PSIRA-compliant officers and presents a deployment confirmation within seconds. SLA timers track response and deployment time.

Tier	Deployment SLA	Process
Standard On-Demand	Officer deployed within 2–4 hours	Full deployment confirmation in <30 seconds
Priority On-Demand	Officer deployed within 60–90 minutes	Dedicated dispatch, client notified at each step
Emergency Deploy	Officer deployed within 30–45 minutes	Command centre escalation, premium rate applies

§ 5.4

Digital Monitoring & Live Operations

The monitoring service provides a real-time operational picture of all active deployments across all venues. Built in Go for high-throughput WebSocket management, it consumes GPS telemetry events from the Guard App and renders a live deployment map for the dispatch team.

- Live map — all active officers plotted on Maplibre GL with real-time GPS updates (30-second intervals)
- Deployment status board — sortable list of all active shifts with clock-in confirmation status
- Incident reporting — officers log incidents from Guard App; appear instantly on monitoring dashboard
- Panic button — one-tap emergency alert from Guard App triggers immediate dispatch escalation
- Digital incident report — structured form (type, severity, parties involved, photos, witnesses)
- Incident escalation matrix — severity-based auto-escalation to supervisor → manager → emergency services
- CCTV integration hooks — external camera system URLs linkable per venue for remote viewing
- Post-shift report auto-generation — summary of shift events, incidents, and anomalies
- Client notification system — venue managers receive push/SMS for any incident at their venue

§ 5.5

ERP — Payroll, Billing & Compliance

The ERP service is built in Java 21 with Spring Boot, providing the financial integrity and audit trail required for a regulated business. It handles payroll calculation, client billing, PSIRA compliance tracking, and the financial reporting required for DFI drawdowns.

Payroll Engine

Calculates officer remuneration based on verified clock-in/out records from blockchain. Deductions (penalties, advances) auto-applied. SARS-compliant payslip generation.

Client Billing

Auto-generates invoices based on completed shifts. Subscription billing for monthly plans. VAT calculation and submission-ready reports.

PSIRA Compliance

Tracks registration status and expiry for every officer. 60/30/14-day renewal alerts. Automatic removal from scheduling pool on expiry.

Financial Ledger

Double-entry accounting ledger. Revenue recognition per shift. Expense tracking for fleet, warehouse, and operational costs.

DFI Reporting

Automated generation of financial reports in formats required by IDC, NEF, SEFA. Revenue per service line, employment metrics, BBBEE compliance.

§ 5.6

Workflow Automation Engine

Powered by Temporal OSS as the durable workflow orchestration engine, the automation layer manages all long-running business processes with guaranteed execution, retry logic, and full audit history.

- Officer onboarding — multi-step workflow from application received to first shift assigned
- Contract renewal — 60-day engagement sequence ending in auto-renewed or escalated contract
- Disciplinary process — structured penalty escalation with documented approval gates
- Client offboarding — graceful wind-down of all active shifts, final invoice, data export
- Monthly payroll run — batch verification of all shift records, payslip generation, bank submission
- PSIRA renewal reminder chain — automated comms to officer at 60/30/14/7/1 days before expiry
- Incident post-processing — structured follow-up workflow after any severity-2+ incident

§ 5.7

Client Portal

The client-facing portal is a Next.js SSR application accessible on web and mobile browser. Venue managers see their entire security operation in one view — live roster, upcoming shifts, incident history, stock levels (Express module), and invoice management.

§ 5.8

Guard Mobile Application

The Guard App is built in React Native (Expo) and is the primary interface for all security personnel. It handles shift confirmation, biometric clock-in/out, incident reporting, panic button, inter-shift communication, and training module access.

- Shift calendar — upcoming assignments with venue name, time, and navigation link
- Biometric clock-in — facial recognition + GPS geofence verification (described in Part VI)
- Incident report form — structured logging with photo capture and severity classification
- Panic button — large, accessible emergency alert with GPS broadcast to dispatch
- Secure messaging — encrypted in-app comms with dispatch team (no personal numbers)
- Training library — video modules, PSIRA content, company policy documents
- Payslip viewer — access to own payslip history and earnings breakdown
- Availability manager — set weekly availability windows, block-out dates
- Push notifications — shift reminders, schedule changes, penalty notifications

§ 5.9

Analytics & Intelligence Dashboard

The analytics service queries ClickHouse for OLAP-grade aggregation performance, serving dashboards that give management deep insight into operational health, revenue performance, and officer productivity — all in real-time.

- Revenue per venue, per region, per service tier — daily/weekly/monthly views
- Deployment density map — heatmap of officer activity across geographic regions
- Officer performance scores — punctuality, incident rate, venue rating, shift completion
- Incident trend analysis — frequency, type, severity, venue clustering
- PSIRA compliance rate — percentage of officers current across the roster
- Demand prediction — projected staffing needs for upcoming weekends and events

- Client retention metrics — churn risk scoring, engagement scores, NPS trends
- Automated client reports — branded PDF delivered monthly to each venue manager

PART VI

BIOMETRIC CLOCK-IN/OUT & PENALTY ENGINE

Facial recognition, geofencing, liveness detection and disciplinary automation

§ 6.1

Facial Recognition Architecture

The biometric verification system is a privacy-first, on-device inference pipeline. Face recognition models run entirely on the officer's mobile device using TensorFlow Lite. Only a cryptographic hash derived from the face embedding is transmitted to the backend — raw biometric data never leaves the device.

Clock-In Verification Flow:

- **Step 1** — Officer opens Guard App and navigates to Clock In for active shift
- **Step 2** — App requests camera permission and activates front camera
- **Step 3** — MediaPipe Face Mesh detects face presence and extracts 468 landmarks
- **Step 4** — Liveness detection module runs (see 6.3) — anti-spoofing check
- **Step 5** — TensorFlow Lite ArcFace model generates 512-dimensional face embedding
- **Step 6** — Embedding compared against enrolled embedding stored on-device (cosine similarity > 0.85 required)
- **Step 7** — GPS coordinates captured and validated against venue geofence polygon ($\pm 50\text{m}$)
- **Step 8** — Biometric hash constructed: `BLAKE3(embedding_hash + officer_id + shift_id + unix_timestamp)`
- **Step 9** — Payload transmitted to Rust Biometric Service via gRPC over TLS
- **Step 10** — Biometric Service validates payload, writes clock-in event to PostgreSQL
- **Step 11** — NATS event published: `deployments.clockin`
- **Step 12** — Blockchain Anchoring Service consumes event, calls `BiometricAnchor.sol.recordClockIn()`
- **Step 13** — Transaction confirmed on Polygon CDK chain — immutable timestamp recorded
- **Step 14** — Dispatch dashboard updates in real-time — officer shown as On-Site

§ 6.2

Geofencing & GPS Verification

Each venue is configured with a GPS polygon geofence in the system. The venue manager or admin defines the boundary via the client portal map interface. Officers must be within the geofence at the time of clock-in for verification to succeed.

- Venue geofence stored as GeoJSON polygon in PostgreSQL with PostGIS extension
- Default tolerance radius: ± 50 metres from venue centroid (configurable per venue)
- Point-in-polygon check performed server-side in Rust Biometric Service for tamper resistance
- GPS accuracy requirement: device must report $< 20\text{m}$ horizontal accuracy
- Spoofed GPS detection: velocity-based checks — impossible travel speed between consecutive pings
- Clock-in rejected if GPS accuracy insufficient — officer prompted to move to better signal area
- All GPS coordinates encrypted in transit and stored with officer consent under POPIA

§ 6.3

Liveness Detection

Anti-spoofing is critical to the integrity of the biometric system. A static photo or video replay must not be able to pass verification. The liveness detection layer uses a combination of passive and active challenge-response methods.

- Passive liveness — texture analysis detects printed photos (moiré patterns, flat lighting)
- Depth analysis — 3D facial geometry estimation prevents flat image spoofing
- Active challenge — randomised blink/turn/nod instruction presented to officer
- Motion vector analysis — detects replayed video by analysing micro-movement patterns
- FaceTec SDK (or open-source equivalent iBeta-tested solution) integrated for certified liveness
- Liveness score threshold configurable per deployment tier (standard / premium)
- Failed liveness attempts logged and flagged for review — 3 failures trigger supervisor notification

§ 6.4

Penalty Engine & Disciplinary Workflow

The penalty engine is an event-driven service that consumes clock-in/clock-out events and calculates compliance against the scheduled shift time. Penalties are automatically applied, recorded on-chain, and feed into the payroll deduction system.

Infraction	Classification	Consequence	Override
0–15 minutes late	Logged Warning	Warning added to record. Venue manager notified via push/SMS.	None
16–30 minutes late	Formal Warning + Pay Deduction	Pro-rata deduction for late period. Written warning generated.	1st offence waivable
31–60 minutes late	Pay Deduction + Client Escalation	Emergency replacement triggered. Client reimbursed for gap. Suspension from this venue.	Manager approval required
60+ minutes / No-Show	Full Disciplinary	Replacement deployed. Full shift pay forfeited. Disciplinary workflow initiated via Temporal.	Director review required
3x Late in 30 days	Performance Review	Automatic scheduling suspension. HR review meeting scheduled.	HR + Manager
5x Late / 2x No-Show	Termination Workflow	Contract termination process initiated. PSIRA notification.	Director sign-off

All penalty events are written to the IncidentLedger smart contract on the Polygon CDK chain, creating an immutable, auditable disciplinary history that cannot be altered by any party — protecting both the company and the officer.

§ 6.5

Biometric Data Privacy & POPIA Compliance

- No raw biometric images are stored on company servers — only cryptographic hashes
- Officers provide explicit, informed consent at onboarding for biometric processing
- Consent records stored with timestamp and version of consent document accepted
- Officers can request deletion of their biometric profile — right to erasure respected
- On-device models and enrolled embeddings encrypted using device secure enclave
- All data transmitted over TLS 1.3 — no biometric data in plaintext
- Data retention policy: biometric hashes retained for 3 years (legal/PSIRA requirement), then purged
- POPIA Information Officer appointed, responsible for breach notification within 72 hours
- Annual third-party privacy audit of biometric processing pipeline
- AWS af-south-1 region ensures data residency within South Africa

PART VII

BLOCKCHAIN INFRASTRUCTURE

Polygon CDK app-chain design, smart contracts and on-chain record types

§ 7.1

Polygon CDK App-Chain Design

Bouncer VIP Express operates a sovereign EVM-compatible blockchain built with Polygon Chain Development Kit (CDK). This dedicated app-chain provides full control over validator set, gas economics, and chain parameters, while maintaining EVM compatibility for Solidity contract development and ethers.js integration.

Parameter	Value
Chain Name	Bouncer VIP Express Chain (BVEC)
Chain ID	Custom (e.g. 91337) — registered on Chainlist
Consensus	PoS with Polygon CDK validator set (initially 3–5 nodes)
Block Time	2 seconds — fast finality for real-time clock-in/out recording
Gas Token	BVX (internal utility token) — zero-cost for system transactions
EVM Compatibility	Full EVM equivalence — all Solidity + ethers.js tools work natively
Data Availability	Polygon AggLayer for DA — posting state diffs to Ethereum for security
RPC Endpoint	Private RPC for internal services + public read-only endpoint for auditors
Block Explorer	Blockscout OSS — self-hosted block explorer for transparency
Validator Nodes	AWS EC2 c5.xlarge instances — 3 initial validators, scalable to 7+

§ 7.2

Smart Contract Architecture

All smart contracts use the transparent upgradeable proxy pattern (OpenZeppelin TransparentUpgradeableProxy) to allow bug fixes without losing on-chain history. A 48-hour timelock is enforced on all upgrades via OpenZeppelin TimelockController.

```
// ShiftRegistry.sol – Immutable Shift Deployment Record
pragma solidity ^0.8.20;

import "@openzeppelin/contracts-upgradeable/access/AccessControlUpgradeable.sol";

contract ShiftRegistry is AccessControlUpgradeable {
    bytes32 public constant RECORDER_ROLE = keccak256("RECORDER_ROLE");

    struct ShiftRecord {
        bytes32 shiftId;           // Internal shift UUID (hashed)
        bytes32 officerId;        // Officer ID (hashed)
        bytes32 venueId;          // Venue ID (hashed)
        uint256 scheduledStart;    // Unix timestamp
        uint256 clockInTime;      // Verified clock-in Unix timestamp
        uint256 clockOutTime;     // Verified clock-out Unix timestamp
        bytes32 biometricHash;     // BLAKE3 hash of biometric payload
        bytes32 geoHash;          // Geofence-verified location hash
        uint8 penaltyTier;        // 0=none, 1=warning, 2=deduction, 3=disciplinary
        bool completed;           // Shift completion flag
    }

    mapping(bytes32 => ShiftRecord) public shifts;

    event ShiftCreated(bytes32 indexed shiftId, bytes32 venueId, uint256 timestamp);
    event ClockInRecorded(bytes32 indexed shiftId, bytes32 biometricHash, uint256 timestamp);
    event ClockOutRecorded(bytes32 indexed shiftId, uint256 duration, uint8 penaltyTier);

    function recordClockIn(bytes32 shiftId, bytes32 officerId,
        bytes32 biometricHash, bytes32 geoHash) external onlyRole(RECORDER_ROLE) {
        shifts[shiftId].clockInTime = block.timestamp;
        shifts[shiftId].biometricHash = biometricHash;
        shifts[shiftId].geoHash = geoHash;
        emit ClockInRecorded(shiftId, biometricHash, block.timestamp);
    }
}
```

§ 7.3

On-Chain Record Types

Contract	Records
ShiftRegistry	Every shift deployment — scheduled time, verified clock-in/out, biometric hashes, penalty tier
BiometricAnchor	Biometric verification events — hash of face embedding + geo + timestamp per clock event
IncidentLedger	Incident reports — severity, type, venue, officer, timestamp, resolution hash
ComplianceVault	PSIRA certificate hashes — registration number hash, expiry date, renewal events
PenaltyRecord	Disciplinary events — infraction type, tier, officer, date, approving manager
DeliveryPoD	Express delivery proof-of-delivery — delivery ID, recipient hash, timestamp, geo hash

Contract	Records
ColdChainAudit	Temperature excursion events — sensor ID, reading, threshold breach, timestamp
ContractMilestone	Client contract events — signing, renewal, termination with authorised signatory hash

§ 7.4

Validator Configuration & Node Setup

The initial validator set consists of three nodes operated by Bouncer VIP Express on dedicated AWS EC2 c5.xlarge instances in the af-south-1 region. As the platform scales, the validator set will be expanded and external institutional validators invited to participate, increasing decentralisation and chain credibility.

```
# Polygon CDK Validator Node — ansible/playbooks/polygon-node.yml
---
- name: Setup Polygon CDK Validator Node
  hosts: polygon_validators
  become: yes
  vars:
    polygon_cdk_version: "latest"
    chain_id: 91337
    data_dir: /opt/polygon-cdk/data
    p2p_port: 30303
    rpc_port: 8545
    ws_port: 8546
  tasks:
    - name: Install Polygon CDK node binary
      get_url:
        url: "https://github.com/0xPolygon/cdk/releases/latest/download/cdk-linux-amd64"
        dest: /usr/local/bin/cdk
        mode: "0755"
    - name: Create data directory
      file: path={{ data_dir }} state=directory
    - name: Write genesis configuration
      template: src=genesis.json.j2 dest={{ data_dir }}/genesis.json
    - name: Start validator node service
      systemd: name=polygon-cdk state=started enabled=yes
```

§ 7.5

Bridge & Interoperability

- Polygon AggLayer integration — state diffs posted to Ethereum for data availability and security
- Bridge contract deployed to Polygon PoS mainnet — enables BVX token bridging if required
- Native bridge UI available to authorized finance team for cross-chain asset movement
- Future: open bridge to enable venue clients to verify their own shift records on public explorers
- Public read-only RPC endpoint — auditors and legal counsel can independently verify records
- Blockscout OSS explorer — self-hosted, no third-party dependency for chain visibility

PART VIII

AI RECRUITMENT & ONBOARDING PIPELINE

Automated credential verification, video analysis and digital onboarding

§ 8.1

Recruitment Funnel Architecture

The AI-powered recruitment pipeline eliminates manual pre-screening bottlenecks, ensuring only qualified, PSIRA-compliant candidates reach human interviewers. The system processes applications end-to-end — from submission through biometric enrolment — with human oversight at critical decision gates.

Stage	Name	Mode	Description
Stage 1	Digital Application	Web/App	Applicant submits personal details, PSIRA number, ID document upload, selfie photo, employment history
Stage 2	AI Pre-Screening	Automated	PSIRA verification, ID OCR + authenticity, face match, criminal cross-reference, completeness scoring
Stage 3	Async Video Interview	Automated + AI	Applicant records video responses; NLP analysis scores verbal content, coherence, red flags
Stage 4	AI Dossier Generation	Automated	System produces candidate report: scores, verified credentials, flagged concerns, recommended questions
Stage 5	Human Interview	Human	Interviewer reviews AI dossier, conducts interview, records decision and notes in platform
Stage 6	Offer & Contract	Automated	Digital offer letter generated, contract signed via DocuSign/in-app, rate card assigned
Stage 7	Digital Onboarding	Automated	Training modules assigned, biometric profile enrolled, PSIRA cert uploaded, scheduling pool activated

§ 8.2

AI Pre-Screening Engine

The pre-screening engine runs a battery of automated checks against each application. Each check produces a sub-score; the composite score determines automatic advancement, human review flag, or rejection.

- PSIRA Number Validation — query PSIRA registry API/scrapper for registration status and grade
- ID Document OCR — extract and verify South African ID number using Tesseract + custom model
- ID Authenticity Check — detect printed or digitally fabricated ID documents (hologram analysis)
- Face Match — compare selfie against ID document photo using ArcFace (threshold: >0.80 similarity)
- Liveness Check on Selfie — ensure selfie is a live capture, not a photo of a photo
- Criminal Record Cross-Reference — query available public records databases and SAPS disclosure
- Employment History Verification — NLP extraction of claimed history, flagging inconsistencies
- Completeness Score — weighted score penalising missing fields or low-quality document uploads
- Composite Score — weighted average triggering: Auto-Advance (>80), Human Review (50–79), Reject (<50)

§ 8.3

Async Video Interview Analysis

Candidates who pass Stage 2 receive a link to record video responses to 5–7 standard questions. The system processes recordings using NLP and audio analysis to score each response independently before a human reviewer sees the application.

Example Interview Questions:

- Tell us about your previous security experience and the environments you have worked in.
 - Describe a situation where you had to de-escalate a conflict at a venue. What did you do?
 - How do you approach access control at a high-volume nightclub entry point?
 - What would you do if you observed a colleague acting aggressively towards a patron?
 - Why do you want to work with Bouncer VIP Express?
-
- Whisper ASR (OpenAI open-source) — high-accuracy speech-to-text transcription of responses
 - BERT-based NLP model — semantic scoring of response content against gold standard answers
 - Keyword presence analysis — security-specific terminology, PSIRA knowledge indicators
 - Red flag detection — violent language, discriminatory statements, inconsistency with application
 - Confidence scoring — speech pattern analysis for fluency, clarity, response coherence
 - Response length analysis — too brief or excessively long responses flagged
 - Video quality check — lighting, audio clarity scored; poor quality flagged for re-recording

§ 8.4

Credential Verification System

-
- PSIRA Certificate — registration number, officer grade (A/B/C/D/E), registration expiry date
 - South African ID — RSA green ID book or Smart ID card, validated check digit algorithm
 - Proof of Address — utility bill / bank statement OCR, address extraction and cross-reference
 - Next-of-Kin Emergency Contact — mandatory, stored encrypted in officer profile
 - Bank Account Verification — account holder name match against officer name (EFT verification)
 - All verified credentials stored as encrypted documents in MinIO with hash references in PostgreSQL
 - Credential hashes anchored to ComplianceVault smart contract for immutable verification record

§ 8.5

Digital Onboarding Flow

- Offer letter generated from template with personalised rate card, shift type, and start date
- Contract signed via in-app digital signature (legally binding under ECT Act, South Africa)
- Biometric enrolment — officer completes 5 facial scans under varied lighting conditions
- ArcFace model generates enrolled embedding stored encrypted on device + hash in backend
- Training module assignment — mandatory PSIRA refresher, company policy, Guard App tutorial
- Test completion required before first shift activation (pass mark: 80%)
- Uniform and equipment checklist — issued items logged against officer profile
- First shift assignment — officer visible in scheduling pool, receives first shift via app

PART IX

EXPRESS — SUPPLY CHAIN OS

Digital and industrial orchestration singularity for the hospitality economy

§ 9.1

Express Platform Overview

Express is a cloud-native supply chain operating system providing the South African nightlife, events, hospitality, fine dining and fast food industries with a single source of truth that integrates directly into venue ERP systems. It functions as the backend, industrial and last-mile layer — eliminating procurement friction, cold chain fragmentation, and last-mile delivery failure through AI-driven orchestration.

Express is structured across three interlocking service layers, all orchestrated through the unified Bouncer VIP Express dashboard and the consumer-facing Express App:

Layer	Name	Description
Layer 1	Warehousing & Cold Chain	Smart 5,000–10,000m² facility on KZN North Coast with multi-temperature zones, IoT sensor grid, and real-time inventory tracking per venue client
Layer 2	Procurement & Supplier Network	Aggregated buying from SAB, Heineken, Distell, Coca-Cola SA — AI demand forecasting, auto-PO generation, consolidated invoicing
Layer 3	Last-Mile Delivery	B2B venue replenishment (refrigerated fleet) + B2C consumer delivery (riders + vans) via Express App — the MrD/Uber Eats equivalent for premium hospitality

§ 9.2

Warehouse Management System

The WMS is the operational core of the Express physical layer. It manages the complete lifecycle of inventory — from supplier goods receipt through to delivery dispatch — across the North Coast facility's multi-temperature storage zones.

Zone	Temperature	Contents
Ambient Zone	18°C–25°C	Dry goods — spirits, mixers, canned goods, paper products, cleaning supplies
Chilled Zone	2°C–8°C	Beer, wine, fresh produce, dairy, pre-made cocktail bases, charcuterie
Frozen Zone	-18°C to -22°C	Frozen food items, ice cream, frozen cocktail ingredients
Bonded Zone	Ambient controlled	High-value spirits and imported beverages requiring customs documentation
Quarantine Zone	Configurable	Stock pending quality inspection, damage assessment, or recall holds

- FIFO/FEFO inventory rotation — First-Expired-First-Out enforced for perishables
- Barcode/QR scanning via warehouse operator app for all goods receipt and dispatch
- Lot tracking — every batch linked to supplier delivery, quality cert, and expiry date
- Real-time stock level per client, per SKU, visible in client portal
- Minimum stock threshold alerts — system auto-generates purchase order recommendation when below threshold
- Bay management — digital map of warehouse floor, bay allocation per client, space utilisation metrics
- Goods receipt workflow — supplier delivery → quality check → zone assignment → inventory updated
- Pick and pack automation — dispatch order generates digital pick list for warehouse operator
- Shrinkage tracking — discrepancy between received and counted stock logged and investigated
- Monthly stock reconciliation — client-facing report of all movements, closing balances, costs

§ 9.3

Cold Chain IoT Architecture

The cold chain IoT layer provides continuous environmental monitoring across all temperature-controlled zones using a network of wireless sensors. All readings are ingested in real-time to InfluxDB and any temperature excursion triggers an immediate multi-channel alert and an immutable on-chain audit record.

```
# Cold Chain IoT Data Flow

Sensors (ESP32 + DHT22 / PT100)  → MQTT Broker (Mosquitto OSS)
                                   → IoT Gateway Service (Go)
                                   → InfluxDB 2 (time-series)
                                   → Grafana Dashboard (real-time)
                                   → NATS: telemetry.coldchain.zone_id
                                   → Alert Engine → SMS/Push/Email
                                   → ColdChainAudit.sol (if excursion)

# Sensor Reading Schema (InfluxDB Line Protocol)
cold_chain,zone=chilled_a,sensor=sens_001 \
  temperature=4.2,humidity=72.1,battery=98 \
  1700000000000000000

# Alert Thresholds (configurable per zone)
chilled_zone:
  warning_high: 8.5°C    # Alert at 0.5° above limit
  critical_high: 9.5°C   # Escalation + on-chain audit
  warning_low: 1.5°C
  critical_low: 0.5°C    # Freezing risk alert
```


§ 9.4

AI Demand Forecasting Engine

The forecasting engine learns each venue's consumption patterns over time and generates predictive stock recommendations, auto-triggering purchase orders when projected stock levels fall below thresholds. The drag-and-drop interface allows venue managers to adjust AI recommendations before confirmation.

- Prophet (Meta) time-series model — captures weekly seasonality, annual patterns, and event spikes
- Features: day of week, public holidays, local events (Durban July, festive season, rugby matches)
- Per-venue model training — individual consumption history trains a bespoke model per client
- Model retraining — Celery worker retrains models weekly on latest 90 days of consumption data
- SKU-level forecasting — beer (by brand), wine (by type), spirits, mixers, food independently modelled
- Auto-PO generation — when predicted stock falls below 5-day buffer, draft PO created for manager approval
- Venue manager drag-and-drop — approve, adjust quantities, or decline AI-generated orders in one tap
- Accuracy tracking — actual vs predicted consumption logged; model performance dashboard for ops team
- Emergency override — venue can bypass AI and place manual on-demand order at any time

§ 9.5

Logistics & Fleet Management

The logistics layer manages the refrigerated fleet responsible for B2B venue replenishment deliveries. Fleet composition will scale with Express revenue, starting with leased vehicles in pre-launch phase before owned fleet acquisition funded through the DFI capital raise.

Vehicle Type	Initial Fleet	Role
3.5-tonne Refrigerated Van	5	Multi-drop venue deliveries — chilled + ambient compartments
1-tonne Refrigerated LDV	8	Smaller venue deliveries, last-minute urgent drops
Refrigerated Motorcycle Box	15	Sub-30-minute premium chilled deliveries within 10km radius
7-tonne Refrigerated Truck	2	Bulk warehouse replenishment from suppliers and distribution

- OSRM (OSS route optimisation) — multi-stop route planning minimising distance and time
- Driver mobile app — route display, stop confirmation, proof-of-delivery capture
- Real-time vehicle GPS tracking — all fleet visible on live operations map
- Temperature monitoring in transit — in-vehicle sensors log and alert during delivery
- Proof-of-delivery — driver captures recipient signature, photo of delivered goods, GPS timestamp
- PoD anchored to DeliveryPoD.sol smart contract — immutable delivery confirmation
- Driver performance scoring — on-time rate, PoD completion, temperature compliance
- Vehicle maintenance scheduling — mileage-based service reminders, breakdown reporting

§ 9.6

Last-Mile Delivery — B2C

The B2C last-mile service is the consumer-facing arm of Express — a premium delivery platform for food, beverages, and alcohol serving consumers in the Greater Durban and KZN coastal markets. Unlike generic delivery apps, Express offers temperature-guaranteed delivery for alcohol and premium food orders.

- Venue menu management — restaurants and bars list their menu on the Express platform
- Consumer Express App (Flutter) — browse menus, order, track delivery in real-time
- Cold delivery guarantee — chilled items delivered at correct temperature or redelivered free
- Alcohol delivery — fully compliant with Liquor Act delivery regulations (age verification via app)
- Group ordering — split payment, multi-address delivery for events
- Loyalty programme — points per order, redeemable for discounts or free delivery
- 18–25% commission on each order — primary B2C revenue driver
- Peak surge pricing — dynamic delivery fee during high-demand periods (Friday/Saturday nights)
- Subscription delivery pass — flat monthly fee for unlimited deliveries within distance bracket

§ 9.7

Supplier Integration Network

Supplier	Products & Integration
SAB Miller (AB InBev)	Beer — Castle, Carling, Corona, Budweiser — bulk procurement integration
Heineken Beverages SA	Beer, cider — Heineken, Amstel, Soweto Gold, Strongbow
Distell Group	Wine, spirits, cider — Nederburg, JC Le Roux, Klipdrift, Savanna
Coca-Cola Beverages SA	Non-alcoholic mixers, energy drinks, soft drink bulk supply
Pernod Ricard SA	Premium spirits — Jameson, Absolut, Chivas, Beefeater
Diageo SA	Premium spirits — Johnnie Walker, Captain Morgan, Tanqueray, Baileys
Local Craft Breweries	KZN craft beer producers — rotating selection, artisan premium tier
Sysco / Bidvest Foodservice	Food items for Express kitchen-integrated venues and B2C food delivery

§ 9.8

Express ERP & Billing

- Storage invoicing — monthly per-client bill based on pallet-days used per zone (ambient/chilled/frozen)

- Delivery fee invoicing — per-delivery charge consolidated on weekly invoice to venue
- Procurement pass-through — goods purchased on behalf of venue invoiced at agreed margin
- SaaS platform fee — monthly fee for dashboard access, AI forecasting, ERP plugin tier
- B2C settlement — daily rider/driver payouts, weekly venue partner revenue share settlement
- DFI draw-down reporting — automated financial reports formatted for IDC, NEF, SEFA requirements
- Tax compliance — VAT-inclusive invoicing, SARS e-filing-ready report exports
- Integration with Bouncer VIP ERP — consolidated group P&L; visible to finance director

PART X

INFRASTRUCTURE & DEVOPS

AWS topology, Coolify orchestration, CI/CD, secrets and observability

§ 10.1

AWS EC2 Deployment Topology

The production infrastructure is deployed on AWS EC2 within the af-south-1 (Cape Town) region, ensuring data residency within South Africa for POPIA compliance. All instances run within a VPC with private subnets for databases and application services, and a public subnet solely for the load balancer and bastion host.

Instance Name	Type	Role
app-primary-01	t3.medium (2vCPU, 4GB)	Coolify orchestrator, primary application services
app-primary-02	t3.medium (2vCPU, 4GB)	Secondary application services, horizontal scaling
compute-ml-01	c5.xlarge (4vCPU, 8GB)	ML model serving, biometric inference, AI recruitment
db-postgres-01	r5.large (2vCPU, 16GB)	PostgreSQL primary — all relational service databases
db-mongo-01	r5.large (2vCPU, 16GB)	MongoDB — incident reports, document storage
db-analytics-01	c5.xlarge (4vCPU, 8GB)	ClickHouse — analytics OLAP + InfluxDB time-series
nats-01 / 02 / 03	t3.medium x3	3-node NATS JetStream cluster
blockchain-01/02/03	c5.xlarge x3	Polygon CDK validator nodes
monitoring-01	t3.medium	Prometheus, Grafana, Loki, Tempo observability stack
bastion-01	t3.micro	SSH bastion host for secure administrative access

§ 10.2

Coolify OSS Orchestration

Coolify OSS serves as the self-hosted PaaS layer, providing a Heroku-like deployment experience on self-managed EC2 instances. It manages container lifecycle, environment variables (via Infisical integration), SSL certificate provisioning, and deployment webhook reception from GitHub Actions.

- Automatic Docker Compose deployment from GitHub repository webhooks
- Blue-green deployment support — zero-downtime releases via traffic switching
- Built-in Traefik integration — automatic subdomain routing and SSL provisioning
- Environment variable management — synced from Infisical OSS at deploy time
- Deployment history — rollback to any previous build in one click
- Resource monitoring — CPU/memory per container visible in Coolify dashboard
- Multiple server support — deploy services across multiple EC2 instances from one UI
- Cron job management — scheduled tasks managed as Coolify jobs

§ 10.3

CI/CD Pipeline Architecture

```
# .github/workflows/deploy.yml – Production Deploy Pipeline
name: Build, Test & Deploy
on:
  push:
    branches: [main]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        service: [crm, scheduling, dispatch, monitoring, erp, biometric]
    steps:
      - uses: actions/checkout@v4
      - name: Run lint + unit tests
        run: cd services/bouncer/${{ matrix.service }}-service && make test

  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run Trivy vulnerability scan
        uses: aquasecurity/trivy-action@master
        with:
          scan-type: fs
          scan-ref: .

  build-and-push:
    needs: [lint-and-test, security-scan]
    runs-on: ubuntu-latest
    steps:
      - name: Build Docker image
        run: docker build -t harbor.bouncervip.com/$SERVICE:$SHA .
      - name: Push to Harbor registry
        run: docker push harbor.bouncervip.com/$SERVICE:$SHA

  deploy:
```

§ 10.4

Secrets Management — Infisical OSS

- Infisical OSS self-hosted on dedicated EC2 instance — no third-party secrets exposure
- All service environment variables pulled from Infisical at container start via Infisical SDK
- AWS Secrets Manager as backup layer — critical DB credentials replicated for DR scenarios
- Secret rotation policy: database passwords rotated every 30 days automatically
- Audit log — every secret access logged with service identity and timestamp
- Environment segregation: development / staging / production namespaces with RBAC
- GitHub Actions secrets: only COOLIFY_WEBHOOK_URL and COOLIFY_TOKEN — no app secrets in GH
- Developer workstations: Infisical CLI for local development — no .env files committed to git

§ 10.5

Traefik Ingress & TLS

```
# traefik/traefik.yml – Production Configuration
api:
  dashboard: true
  insecure: false # Dashboard behind BasicAuth

entryPoints:
  web:
    address: ":80"
    http:
      redirections:
        entryPoint:
          to: websecure
          scheme: https
  websecure:
    address: ":443"
    http:
      tls:
        certResolver: letsencrypt

certificatesResolvers:
  letsencrypt:
    acme:
      email: devops@bouncervip.co.za
      storage: /letsencrypt/acme.json
      tlsChallenge: {}

providers:
  docker:
    exposedByDefault: false
  file:
    directory: /etc/traefik/dynamic/
```

§ 10.6

Kong OSS API Gateway

- JWT validation plugin — validates all incoming API requests against Keycloak-issued tokens
- Rate limiting plugin — per-client, per-route limits (e.g. 1000 req/min for venue portals)
- Request/response logging — all API traffic logged to Loki via Kong logging plugin
- CORS plugin — controlled cross-origin policy per service route
- Bot detection — basic bot pattern filtering at gateway level
- Route versioning — /api/v1/ and /api/v2/ routing for backward-compatible API evolution
- Health check routes — /health endpoint exposed per service for uptime monitoring
- Admin API secured — Kong Admin API accessible only from bastion host via private network

§ 10.7

NATS JetStream Event Bus

NATS JetStream is deployed as a 3-node cluster across nats-01, nats-02, nats-03 EC2 instances, providing R=3 replication for all critical streams. This ensures no event loss even if a single node fails.

§ 10.8

Grafana Observability Stack

The full Grafana LGTM stack (Loki, Grafana, Tempo, Mimir/Prometheus) is deployed on monitoring-01. Custom dashboards are maintained in code (Grafana provisioning) and version-controlled in the monorepo under infra/monitoring/.

- Platform Overview — request rates, error rates, latency (p50/p95/p99) per service
- Deployment Operations — active shifts, officers on-site, pending clock-ins, incidents
- Biometric Verification — clock-in success rate, liveness failure rate, geofence violations
- Cold Chain Health — real-time temperature per zone, excursion history, sensor battery
- Express Logistics — active deliveries, on-time rate, fleet GPS positions
- Blockchain Health — block production rate, transaction throughput, validator uptime
- Business Metrics — revenue per day, venues active, bookings, churn indicators
- Security Events — failed auth attempts, rate limit hits, anomaly alerts

PART XI

DEVELOPMENT DIRECTIVES

Engineering principles, coding standards, API design and testing strategy

§ 11.1

Engineering Principles

You Build It, You Run It

Each squad owns their services end-to-end — development, testing, deployment, and on-call. No handoffs to a separate ops team.

API Contract First

Proto files and OpenAPI specs are written and reviewed before implementation begins. No ad-hoc API changes without contract update.

Test Coverage Gate

No service merged to main without: unit tests (>80% coverage), integration tests for all gRPC handlers, and at least one E2E scenario.

Security by Default

OWASP Top 10 review at every PR. No secrets in code. mTLS between all services. RBAC validated at service boundary, not just gateway.

Observability Day One

Every service ships with structured logs, Prometheus metrics, and OpenTelemetry traces from the first commit. Not retrofitted later.

Immutable Infrastructure

No SSH changes to running production servers. All changes via Terraform, Ansible, or Coolify deployment. Servers are cattle, not pets.

Twelve-Factor App

All services follow 12-factor principles: config from environment, stateless processes, explicit dependency declaration, log to stdout.

Fail Fast, Recover Faster

Circuit breakers on all outbound calls. Health checks on all services. Graceful degradation over hard failure. MTTR < 15 minutes target.

§ 11.2

Coding Standards per Language

TypeScript

Strict mode enabled. ESLint with typescript-eslint ruleset. Prettier formatting. Zod for runtime validation. No any types permitted.

Go

gofmt enforced in CI. golangci-lint with errcheck, staticcheck, govet enabled. All errors explicitly handled — no underscore discards. Interfaces defined at consumer.

Python

Black formatter. Ruff linter. mypy for type checking (strict mode). Pydantic models for all API request/response schemas. No bare except clauses.

Java

Checkstyle with Google Java Style. SpotBugs for static analysis. Lombok for boilerplate reduction. Records for DTOs where possible. Spring profiles for env config.

Rust

clippy with deny(warnings). rustfmt formatting. No unwrap() in production code — all Results explicitly handled. Tokio for async, tonic for gRPC.

Elixir

mix format enforced. Credo for code quality analysis. Dialyzer for type checking. No raw Task.async without supervision tree. Phoenix contexts for domain boundaries.

Solidity

Solhint linter. Natspec documentation on all public functions. Slither static analysis in CI. No tx.origin usage. Checks-Effects-Interactions pattern enforced.

§ 11.3

API Design Standards

- gRPC: use proto3 syntax only; no required fields; use field masks for partial updates; version all packages (bouncer.shift.v1)
- REST (external via Kong): noun-based resource paths (/api/v1/venues/{id}/shifts); HTTP verbs strictly semantic; 4xx/5xx error schemas standardised across all services
- Pagination: cursor-based pagination for all list endpoints — no offset pagination for large datasets
- Idempotency: all POST/PUT endpoints accept X-Idempotency-Key header; duplicate requests return same response
- Versioning: URI versioning (/v1/, /v2/) for external APIs; proto package versioning for internal gRPC
- Error format: standard JSON error envelope {code, message, details[]} across all REST responses
- Documentation: every external endpoint documented in OpenAPI 3.1 spec, auto-generated from code annotations
- Deprecation: minimum 6-month deprecation notice for any breaking API change; sunset header on deprecated routes

§ 11.4

Database-per-Service Strategy

No two services share a database schema. Cross-service data access is achieved exclusively through gRPC API calls or NATS event consumption. This enforces true service independence and allows each database to be scaled, migrated, or replaced independently.

- Schema migrations managed per-service using Flyway (Java), golang-migrate (Go), Alembic (Python), or Ecto (Elixir)
- All migrations are forward-only in production — no rollback migrations (use compensating migrations instead)
- Database credentials rotated via Infisical on 30-day cycle — services restart automatically with new credentials
- Read replicas provisioned for analytics-heavy services (ERP, Analytics) to prevent OLAP queries impacting OLTP
- Connection pooling via PgBouncer for all PostgreSQL connections — max pool size configured per service
- All database access via parameterised queries or ORM — no string interpolation in queries

§ 11.5

Testing Strategy

Test Type	Scope	Tools
Unit Tests	Every function/method in isolation. Mocks for all external dependencies. Coverage gate: >80%.	Jest (TS), go test, pytest, JUnit 5, Cargo test, ExUnit, Hardhat
Integration Tests	gRPC handler testing against real database (test containers). NATS consumer testing.	Testcontainers for ephemeral DB instances per test run
Contract Tests	Proto contract compliance tests — ensure generated clients match server implementation.	Pact for consumer-driven contract testing between services
E2E Tests	Critical user journeys: officer clock-in flow, booking creation, invoice generation.	Playwright for web app E2E, Detox for React Native app
Load Tests	Simulate 500 concurrent on-demand booking requests. Biometric service under 100 simultaneous clock-ins.	k6 OSS load testing framework
Smart Contract Tests	All contract state transitions. Fuzz testing for edge cases. Gas usage assertions.	Hardhat (TypeScript) + Foundry Forge (Solidity fuzz)
Security Tests	OWASP ZAP DAST scan on all external endpoints. Dependency vulnerability scan on every PR.	OWASP ZAP, Trivy, Snyk OSS

§ 11.6

Security Development Lifecycle

- Threat modelling conducted for every new service before development begins (STRIDE methodology)
- OWASP Top 10 checklist reviewed at each PR for services with external interfaces
- Trivy container scan must pass (no Critical/High CVEs) before image pushed to Harbor registry
- Slither static analysis runs on every Solidity change in CI — any HIGH findings block merge
- Penetration testing: annual third-party pentest of all external-facing APIs and mobile applications
- Dependency updates: Renovate bot configured to auto-create PRs for dependency updates weekly
- Incident response plan: documented runbook for security incidents, breach notification procedure under POPIA
- Biometric data audit: quarterly review of biometric hash storage, access logs, and consent records

PART XII

PHASED ROLLOUT ROADMAP

From launch through national expansion — milestone-driven development plan

§ 12.1

Phase 1 — Core Security Launch (Months 0–3)

Objective: Generate revenue, establish brand, validate market.

- Sign 10–15 anchor venue clients on Florida Road, Umhlanga, and Morningside corridors
- Deploy Bouncer VIP web platform: CRM, scheduling, client portal (read-only), basic monitoring
- Guard App v1: shift calendar, GPS clock-in (no biometrics yet), incident reporting
- Manual PSIRA compliance tracking (automated in Phase 2)
- ERP v1: payroll calculation from manual timesheets, basic invoice generation
- Establish officer pool of 50+ PSIRA-registered, vetted security personnel
- Achieve R200,000+ MRR by end of Month 3
- Infra: 3 EC2 instances, basic Coolify setup, Traefik, PostgreSQL, Redis

§ 12.2

Phase 2 — Biometrics, Blockchain & AI (Months 4–7)

Objective: Differentiate through technology, build the moat.

- Deploy Polygon CDK app-chain (testnet Month 4, mainnet Month 5)
- Launch biometric clock-in/out on Guard App — facial recognition + geofence
- Activate Penalty Engine — automated lateness detection and discipline workflows
- Deploy AI Recruitment Pipeline — reduce manual screening time by >70%
- PSIRA compliance automation — expiry tracking, automated alerts, scheduling pool management
- Blockchain-verified shift records marketing — pitch to venues as legal evidence capability

- Expand to 30–40 venue clients across Greater Durban
- Launch Analytics Dashboard v1 — venue-facing insight reports
- Target: R500,000+ MRR by end of Month 7

§ 12.3

Phase 3 — Express Launch (Months 8–14)

Objective: Launch supply chain OS, secure DFI funding, build warehouse facility.

- Month 8: Secure R280M–R450M capital raise (ABSA, Standard Bank, IDC, NEF combination)
- Month 8: Begin warehouse facility construction / fit-out on KZN North Coast
- Month 9: Launch Express WMS and cold chain IoT monitoring on leased pilot facility
- Month 9: Onboard first 10 Express venue clients — Storage as a Service
- Month 10: Launch procurement aggregation — first supplier agreements with SAB, Heineken
- Month 11: B2B refrigerated delivery fleet operational (initial leased vehicle fleet)
- Month 12: AI demand forecasting engine live — venue managers see first AI-generated POs
- Month 13: Express App (Flutter) launched — B2C last-mile delivery activated
- Month 14: Full warehouse facility operational — all cold chain zones live
- Target: R1.5M+ combined MRR (Bouncer VIP + Express) by Month 14

§ 12.4

Phase 4 — National Expansion (Months 15–36)

Objective: Scale to Johannesburg, Cape Town, and platform licensing.

- Month 15: Expand security operations to Johannesburg CBD and Sandton nightlife corridor
- Month 18: Cape Town expansion — Long Street, V&A; Waterfront, Sea Point venue corridor
- Month 20: Second warehouse facility (Gauteng) — enables national cold chain coverage
- Month 22: Launch Bouncer VIP Platform white-label licensing to other security companies
- Month 24: Express SaaS licensing — sell platform access to hospitality operators without storage service
- Month 30: Port Elizabeth and Bloemfontein market entry
- Month 36: Pan-African expansion assessment — Zimbabwe, Mozambique, Zambia feasibility
- Target: R10M+ MRR by Month 36, 500+ venue clients nationally

Phase	Timeline	Focus	Revenue Target	Key Milestones
Phase 1	M0–M3	Security Launch	R200K MRR	Venue clients signed, platform live
Phase 2	M4–M7	Tech Differentiation	R500K MRR	Biometrics, blockchain, AI recruitment live
Phase 3	M8–M14	Express Launch	R1.5M MRR	Warehouse operational, B2C live

Phase	Timeline	Focus	Revenue Target	Key Milestones
Phase 4	M15–M36	National Scale	R10M+ MRR	Multi-city, white-label, pan-Africa prep

PART XIII

INVESTOR BRIEF

Investment thesis, market opportunity, financial projections and funding strategy

§ 13.1

Investment Thesis

Bouncer VIP Express operates at the intersection of three large, underserved markets: private security services (R65B/year), cold chain logistics (R12B/year), and on-demand delivery (R8–10B/year). No incumbent competitor in South Africa has built a technology platform that integrates these three verticals under one operating system.

The company has a capital-efficient entry strategy — the security services business generates immediate cash from Day 1 with minimal capex (staffing and software). This revenue and the operational credibility it builds funds the larger Express infrastructure raise. Investors and lenders are presented with a de-risked opportunity: a technology business with proven cash flow seeking infrastructure capital.

- First-mover advantage in technology-enabled nightlife security in South Africa
- Blockchain verification creates legal defensibility unmatched by any competitor
- Network effects: more venues → more officers trained → better AI models → better service
- Three revenue streams operating simultaneously (security + logistics + SaaS)
- White-label licensing potential: Bouncer VIP platform sold to 1,000+ SA security companies
- Express creates a physical infrastructure moat — cannot be replicated in 18 months
- Management team with deep KZN market relationships and technology execution capability

§ 13.2

Market Opportunity

Market	Total Size	Segment	Addressable
SA Private Security Industry	R65 billion/year	Nightlife/Events sub-segment (est.)	R8–12 billion/year
SA Cold Chain Logistics	R12 billion/year	Hospitality-specific (est.)	R2–3 billion/year
SA On-Demand Delivery	R8–10 billion/year	KZN addressable (est.)	R800M–1.2B/year
KZN Nightlife Venues (TAM)	600–800 venues	Greater Durban SAM (Year 1)	300–500 venues

§ 13.3

Revenue Model & Financial Projections

Revenue Streams Across the Group:

Revenue Stream	Rate	Activation
Security Subscriptions	R1,300–R3,600+/night per venue	Immediate (Month 1)
Platform SaaS (BVP)	R890–R6,500/month per venue	Month 3+
Biometric Premium Tier	+R350/month per venue	Month 6+
Storage as a Service	R3,500–R16,500+/month per venue	Month 9+
Logistics Delivery Fees	R180–R2,000 per delivery	Month 11+
Procurement Margin	5–12% on goods GMV	Month 10+
B2C Delivery Commission	18–25% per consumer order	Month 13+
Express SaaS Platform	R890–R6,500/month per venue	Month 9+
White-Label Licensing	R15,000–R45,000/month per licensee	Month 22+

Conservative Financial Projections:

Period	Venue Count	Revenue	EBITDA	EBITDA Margin
Year 1 (Security live, Express pre-launch)	50 venues	R8.5M	R2.1M	25%
Year 2 (Express operational, 120 venues)	120 venues	R38M	R11.4M	30%
Year 3 (Multi-city, white-label active)	280 venues	R105M	R37.8M	36%
Year 4 (National + regional, 500 venues)	500 venues	R240M	R96M	40%
Year 5 (Platform licensing + pan-Africa)	800 venues	R480M	R211M	44%

§ 13.4

Capital Requirements

Phase 1–2 Capital (Technology + Operations): R15M–R25M

Item	Cost Range
Platform Development (full tech stack)	R8M–R12M
Initial Officer Recruitment & Training	R2M–R3M
Operations Setup (offices, equipment, vehicles — leased)	R2M–R4M
Working Capital Buffer (6 months runway)	R3M–R6M

Phase 3 Capital (Express Infrastructure): R280M–R450M

Capital Item	Estimated Cost
Land Acquisition (North Coast, 5,000–10,000m²)	R15M–R35M
Warehouse Construction & Fit-Out	R80M–R140M
Cold Chain Refrigeration Plant & IoT	R40M–R70M
Fleet Acquisition (trucks, vans, bikes)	R35M–R60M
Technology Platform (Express modules)	R15M–R25M
Working Capital & Operational Buffer	R30M–R50M
Regulatory, Legal & Compliance Setup	R8M–R15M
Contingency (10%)	R22M–R39M

§ 13.5

Funding Strategy — Banks & DFIs

The capital strategy is structured in two tranches aligned to the phased development plan. Phase 1–2 technology capital is raised through equity (founder + angel investment). The Phase 3 infrastructure raise targets a combination of commercial bank debt and Development Finance Institution (DFI) facilities.

Institution	Type	Target Amount	Rationale
ABSA Corporate & Investment Banking	Commercial Debt	R80M–R120M	Infrastructure-backed term loan. Strong SME and infrastructure lending mandate.
Standard Bank Business Banking	Commercial Debt	R60M–R100M	Cold chain and logistics infrastructure. Asset-backed facility against warehouse + fleet.

Institution	Type	Target Amount	Rationale
Industrial Development Corporation (IDC)	DFI Equity/Debt	R60M–R100M	Logistics, cold chain, and manufacturing infrastructure. Job creation mandate.
National Empowerment Fund (NEF)	DFI Equity	R30M–R60M	Black-owned business empowerment. Technology + infrastructure category.
SEFA	DFI Debt	R15M–R25M	SME growth funding for technology development tranche.
Founder / Angel	Equity	R15M–R25M	Phase 1–2 technology development capital.

§ 13.6

Risk Analysis & Mitigation

Risk	Rating	Mitigation
Regulatory — Liquor Distribution Licence	HIGH	Engage specialist liquor licence attorney from Month 1. 18-month lead time planned.
Cold Chain Liability — Stock Loss	MEDIUM	Comprehensive cold chain insurance. Redundant refrigeration systems. 24/7 IoT monitoring.
Officer Reliability & No-Shows	MEDIUM	Penalty engine + backup pool protocol. Target 120% roster capacity vs projected demand.
Technology Execution Complexity	HIGH	Phased development — MVP first, features layered in. Senior polyglot engineering team.
DFI Funding Timeline	MEDIUM	Early engagement (Month 3). Security revenue as proof-of-market. Multiple lender approach.
Competitor Response	LOW-MEDIUM	Blockchain + biometric moat takes 18+ months to replicate. First-mover critical mass.
POPIA / Biometric Compliance	MEDIUM	POPIA Information Officer appointed. Privacy-by-design. No raw biometrics stored.
Market Adoption (Venues)	MEDIUM	Anchor client strategy with 90-day trial offers. Differentiated tech platform as hook.

§ 13.7

Competitive Moat

- Blockchain-verified operations — legally defensible shift and incident records; no competitor offers this
- Biometric verification — officer presence 100% verified; eliminates ghost employee fraud
- AI-powered recruitment pipeline — lower cost per hire, higher quality intake than manual screening
- Integrated supply chain OS — security + procurement + cold chain + delivery in one platform
- Physical warehouse infrastructure — 5,000–10,000m² owned facility cannot be rapidly replicated

- Aggregated buying power — bulk supplier agreements drive pricing advantages unavailable to individual venues
- Data network effect — every deployment, delivery, and sensor reading improves AI models for all clients
- White-label licensing potential — Bouncer VIP platform sold to competitors, monetising the moat

§ 13.8

Exit Strategy

The company is built with multiple credible exit pathways available at different stages of growth, providing investors with clear liquidity options.

Exit Route	Description
Strategic Acquisition (Year 4–6)	Large private security group (G4S, Fidelity, Stallion) acquires platform + operations. Estimated 6–8x revenue multiple.
Pan-African Platform Acquisition (Year 5–7)	Pan-African logistics or delivery conglomerate (Bolloré Logistics, DHL Africa) acquires Express layer. 8–12x EBITDA.
JSE Listing (Year 6–8)	Public listing on JSE AltX or Main Board. Technology + infrastructure profile suitable for institutional investment.
Management Buyout (Year 5+)	Management team buys out early investors at pre-agreed multiple if strategic sale not preferred.

For investor enquiries, partnership discussions, or DFI engagement, contact:
invest@bouncervip.co.za | +27 (0) 31 000 0000 Bouncer VIP Express (Pty) Ltd · Greater Durban, KwaZulu-Natal, South Africa